

हेलो! आपका स्वागत है **Unit 13: DOM (Document Object Model)** की complete study class में।
मैंने आपके workspace में, unit 13 DOM folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter13_dom.md
- Mini Project Code File: dom_project.html
- Practice Exercises Sandbox: dom_practice.html

चलिए अब DOM Structure, selections, element manipulations, styles aur forms validations features ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: DOM STRUCTURE & TREE

DOM humare JavaScript code ko pure HTML document elements ko dynamically select aur change karne ki capability deta hai.

1. DOM Kya Hai?

What is it? DOM (Document Object Model) ek tree-like logical programming API schema hai jo browser level par physical HTML code files ko structural objects container mein model karta hai.

How HTML becomes DOM? Browser jaise hi webpage compile parsing complete karta hai, browser memory block ke andar document elements nodes hierarchies structure sets mapping register kar leta hai jise we represent as document.

DOM Tree:

html

Document (Root)

```
└─ <html>
  └─ <head>
    └─ <body>
      └─ <h1> (Heading element node)
        └─ <p> (Paragraph text node)
```

Node vs Element Difference:

- **Node:** DOM tree structure ka har point node kehlata hai (e.g. comment lines, white space text nodes).

- **Element:** tags representations nodes (like <div> or <p>). All Elements are Nodes, but all Nodes are not Elements.
-

2. selectors Methods

What is it? HTML elements select resources.

- **getElementById("id"):** Queries elements by ID matching name. Returns 1 object pointer.
- **querySelector("selector"):** CSS styling syntax parser runs, retrieves first matched element reference.
- **querySelectorAll("selector"):** returns static collection array-like NodeList containing all matches.

Simple example

javascript

```
let header = document.getElementById("title");  
let firstBox = document.querySelector(".box");  
let allParagraphs = document.querySelectorAll("p");
```

PART 2: CONTENT & ATTRIBUTES MANIPULATIONS

1. textContent vs innerHTML

What is it? HTML text writing variables updates elements updates targets.

- **textContent:** processes and writes plain texts strictly. HTML markup code is written literally as string block.
- **innerHTML:** parses tags strings compile into actual HTML layout nodes.

Simple example

javascript

```
let container = document.querySelector("#box");  
container.textContent = "<strong>Hello</strong>"; // Shows tag strings literally  
container.innerHTML = "<strong>Hello</strong>"; // Shows bold formatted text
```

2. Attributes Management

What is it? Tags metadata attributes updates.

- **getAttribute("name")**: returns active values.
- **setAttribute("name", "value")**: writes updates properties.
- **removeAttribute("name")**: deletes.

Simple example

javascript

```
let anchor = document.querySelector("a");  
console.log(anchor.getAttribute("href"));  
anchor.setAttribute("href", "https://google.com");  
anchor.removeAttribute("href");
```

3. classList API

What is it? Style classifications selectors toggles.

- **add("className")**: appends styles.
- **remove("className")**: deletes.
- **toggle("className")**: dynamically adds if missing, removes if present.
- **contains("className")**: verifies if active (returns true/false).

Simple example

javascript

```
let box = document.querySelector("div");  
box.classList.add("highlight");  
box.classList.toggle("active");
```

4. Element CRUD (Create, Append, Delete)

What is it? Creating and deleting elements dynamically.

- **createElement("tag")**: memory creation tag elements.
- **append()**: appends multiple elements or text strings.

- **remove():** deletes element.

Simple example

javascript

```
let parentBox = document.getElementById("holder");  
  
let newTag = document.createElement("p");  
  
newTag.textContent = "Appended item";  
  
parentBox.append(newTag);  
  
newTag.remove();
```

PART 3: STYLES, FORMS & VALIDATIONS

1. CSS styling via JS

Syntax: `element.style.propertyName = "value";` (camelCase formats).

Simple example

javascript

```
let txt = document.querySelector("p");  
  
txt.style.color = "blue";  
  
txt.style.fontSize = "20px";
```

2. Forms submissions & Validations

Forms submit validations coordinates:

javascript

```
let form = document.querySelector("form");  
  
let userVal = document.querySelector("input");  
  
form.addEventListener("submit", (event) => {  
    event.preventDefault(); // stops page reloads  
  
    let val = userVal.value.trim();  
  
    if (val.length < 3) {  
        console.log("Validation mismatch error: too short input value!");  
    }  
});
```

```
}  
});
```

PART 4: DOM MINI PROJECT

Project Title: Task Checklist Builder Run the complete HTML project file dom_project.html directly inside your web browser.

It covers:

1. Form submission and input capture using value.trim().
 2. Input verification checks to display errorDiv.
 3. DOM dynamic creation using createElement("li") and class toggling using classList.toggle("checked") on click event triggers.
 4. Child element removal logic on clicking delete button.
-



QUICK REVISION SUMMARY

- **Selectors:** getElementById selects by ID; querySelector returns first CSS selector match; querySelectorAll returns static NodeList.
 - **Content:** textContent ignores HTML format parser; innerHTML executes HTML tags parser.
 - **classList:** add() appends styles class; remove() deletes class; toggle() toggles presence.
 - **CRUD:** createElement in-memory creates; append appends elements; remove clears.
 - **Styling:** inline styles are written using camelCase format.
 - **Forms:** value reads input state; preventDefault() stops page reload.
-



IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Document Object Model:** Programming interface mapping webpage document structures as trees.
2. **Node vs Element:** Node is a generic term representing any DOM tree coordinate; Element is a specific tag node type.

3. **Event delegation:** Dynamic target listener assignments to parents using event bubbling attributes.
 4. **NodeList:** Collection array-like list of elements returned by selector queries.
 5. **Form Revalidation:** Halting default submit reload operations using `preventDefault()`.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between `textContent` and `innerHTML`?

Ans: `textContent` reads/writes raw text content. `innerHTML` parses input string tags, rendering actual HTML formats.

Q2. Difference between `querySelector()` and `querySelectorAll()`?

Ans: `querySelector()` returns the first matching element object or null. `querySelectorAll()` yields a static array-like `NodeList` containing all matching element references.

Q3. Explain Node vs Element.

Ans: Node is a generic base interface representing any node in the DOM tree (like text or comment). Element is a specific node interface representing HTML tags.

Q4. What is `event.preventDefault()`?

Ans: It stops the browser's default action associated with the event (e.g. stops page reload on form submit).

Q5. How does `classList.toggle()` work?

Ans: It adds the class if it does not exist on the element, and removes it if it does.

Q6. Predict output:

```
javascript
```

```
let container = document.createElement("div");
```

```
console.log(container.innerHTML);
```

Ans: `""` (empty string). Created in-memory elements start with empty content.

Q7. How does event delegation work?

Ans: Attaching event listener to a parent node instead of separate listeners on multiple child elements, utilizing event bubbling.

Q8. What does `element.style` modify?

Ans: It applies inline styles directly to the element, overriding values specified in CSS files.

Q9. Predict output:

```
javascript  
  
let text = document.querySelector("#title");  
  
// if ID title does not exist  
  
console.log(text);
```

Ans: null.

Q10. Difference between append() and appendChild()?

Ans: append() can attach multiple elements and plain text strings. appendChild() only accepts a single Node object.

Q11. Predict output:

```
javascript  
  
let li = document.createElement("li");  
  
li.innerText = "item";  
  
console.log(li.parentNode);
```

Ans: null. Newly created elements do not have parent links until they are appended to the document.

Q12. Predict output:

```
javascript  
  
let tag = document.querySelector("input");  
  
// user types: " Ravi "  
  
console.log(tag.value.trim().length);
```

Ans: 4. Trim removes edge whitespace characters.

Q13. How do you remove an attribute from an HTML element?

Ans: element.removeAttribute("attributeName").

Q14. What does classList.contains() return?

Ans: Boolean true or false indicating if the class is present on the element.

Q15. Is NodeList a real Array?

Ans: No. It is an array-like object. It supports `forEach()` but lacks array methods like `map()` or `filter()`.

Q16. Predict output:

javascript

```
let heading = document.querySelector("h1");  
heading.style.background = "red";
```

Ans: Sets inline style background-color equivalent to red on the page element.

Q17. What is the root object of the DOM?

Ans: The document object.

Q18. Predict output:

javascript

```
let div = document.createElement("div");  
div.classList.toggle("test");  
console.log(div.classList.contains("test"));
```

Ans: true. Toggle adds the class since it was not present.

Q19. How do you stop event bubbling?

Ans: By calling `event.stopPropagation()` inside the event listener.

Q20. Predict output:

javascript

```
let element = document.getElementById("nonExistant");  
console.log(element);
```

Ans: null.



PRACTICE QUESTIONS

Question 1: Class Toggling Button

- **Question:** Write code that changes background color to yellow by toggling class on button click.
- **HTML:** `<div id="box">Content</div><button id="toggleBtn">Toggle</button>`

- **JavaScript:**

```
javascript
```

```
const box = document.getElementById("box");

const toggleBtn = document.getElementById("toggleBtn");

toggleBtn.addEventListener("click", () => {

    box.classList.toggle("highlight");

});
```

- **Explanation:** Toggles class highlight (yellow background in CSS) on the target box.

Question 2: Read Input and Update Heading text

- **Question:** Create input inputting name variables updates h1 element tags text content.
- **HTML:** <h1 id="title">User</h1><input id="nameInput"><button id="btn">Update</button>
- **JavaScript:**

```
javascript
```

```
const title = document.getElementById("title");

const nameInput = document.getElementById("nameInput");

const btn = document.getElementById("btn");

btn.addEventListener("click", () => {

    let cleanVal = nameInput.value.trim();

    title.textContent = cleanVal !== "" ? cleanVal : "User";

});
```

- **Explanation:** Reads input value, trims whitespace, and updates textContent.

Question 3: Form submission validation constraints

- **Question:** Stop form reloading and alert if input length is less than 3.
- **JavaScript:**

```
javascript
```

```
const form = document.querySelector("form");

const input = document.querySelector("input");
```

```
form.addEventListener("submit", (e) => {  
    e.preventDefault();  
    if (input.value.trim().length < 3) {  
        console.log("Failed: Name too short!");  
    }  
});
```

- **Explanation:** Calls `preventDefault()` to cancel page reload and applies condition checking to input length.

9:48 PM